

Lab: Design a Simple Online Booking System Using UML Diagrams

Learning objectives

By the end of this lab, you will be able to:

- Create UML diagrams to model an online booking system's architecture and workflows
- Identify system requirements using actors, use cases, and class structures
- Illustrate system behavior using sequence and activity diagrams

Prerequisites

- Review the following videos:
 - **Overview of Unified Modeling Language (UML) and Its Importance**
 - **Class and Sequence Diagrams**
 - **Activity and State Diagrams**
- Go through the **Reading: Use Case Diagrams**
- Complete your tasks using any diagramming software.
 - **For UML diagrams:** Use the free version of Lucidchart or Draw.io to complete the diagramming tasks in this lab. If you upgrade to a paid plan, you will be responsible for any associated charges.
- **Basic knowledge required:**
 - Understanding of object-oriented programming concepts
 - Familiarity with system analysis and design principles
 - Basic knowledge of UML notation and diagram types

Scenario

MovieGo is a growing online movie ticket booking platform offering customers the ability to browse movies, select showtimes, book tickets, and make payments through its website. Administrators use the system to manage movie schedules and monitor bookings. A secure third-party payment gateway handles financial transactions.

At its core, the system must support five key activities:

- Movie browsing
- Ticket booking
- Payment processing
- Schedule management

- Booking visibility for admins

However, for the system to be effective, it must clearly represent how users interact with the platform and how internal components relate to one another. This lab guides you through creating four foundational UML diagrams to model system behavior, structure, and interaction. Your diagrams will help translate real-world business needs into clear, actionable visual representations that can guide future development and documentation efforts.

Part 1: Create a use case diagram

Why this matters:

A use case diagram helps define what the system should do by identifying who interacts with it and what actions they perform. This provides a high-level view of system functionality.

Task:

Create a use case diagram for the online booking system

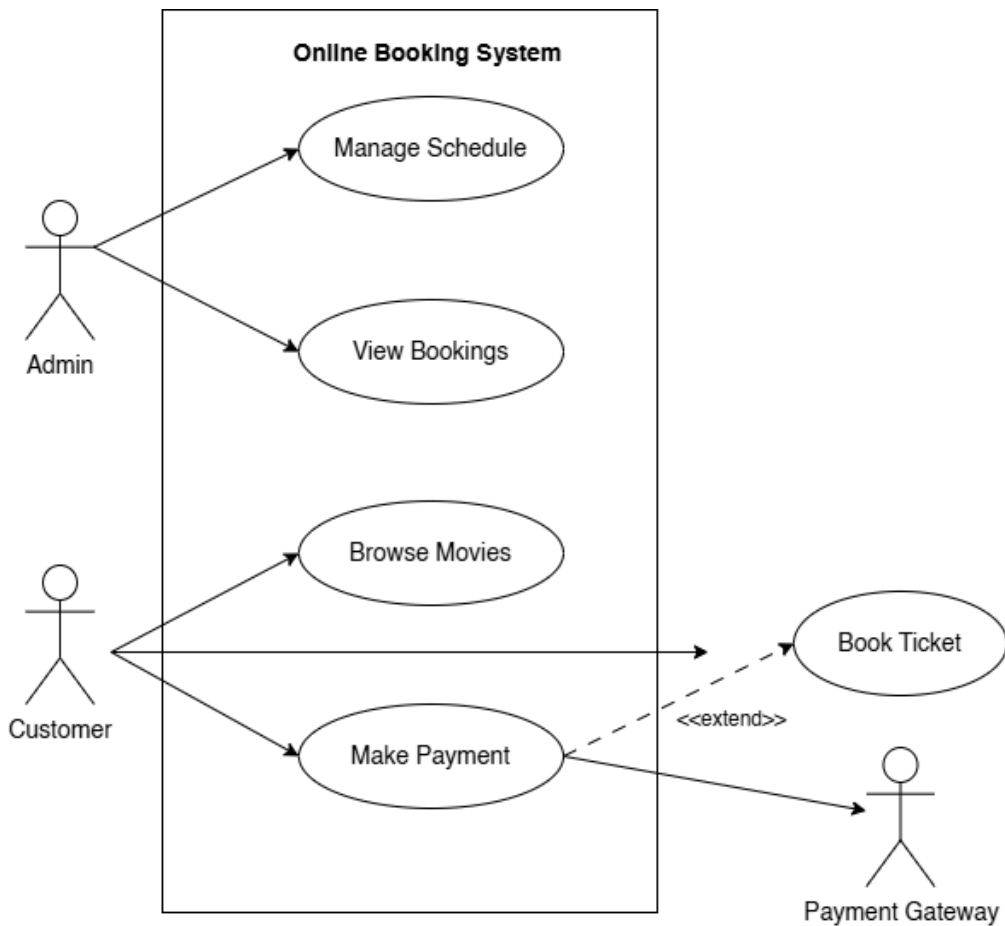
Instructions:

- Identify all actors involved: Customer, Admin, Payment Gateway
- Define the main use cases: Browse Movies, Book Ticket, Make Payment, Manage Schedule, View Bookings
- Draw relationships between actors and use cases, including include/extend where relevant
- Enclose all use cases within the "Online Booking System" boundary

Use these elements in your use case diagram:

- Actors: Customer, Admin, Payment Gateway
- Use cases:
 - Customer: Browse Movies, Book Ticket, Make Payment
 - Admin: Manage Schedule, View Bookings
 - Payment Gateway: interacts with Make Payment (extends from Book Ticket)
- Relationships:
 - Customer ↔ Browse Movies, Book Ticket, Make Payment
 - Admin ↔ Manage Schedule, View Bookings
 - Payment Gateway ↔ Make Payment (extends Book Ticket)
 - System boundary labeled as "Online Booking System"

▼ [Click here to view the solution:](#)



Part 2: Create a class diagram

Why this matters:

A class diagram shows the static structure of the system—its entities, their attributes, and relationships. This provides the foundation for database and object-oriented design.

Task:

Create a class diagram for the system

Instructions:

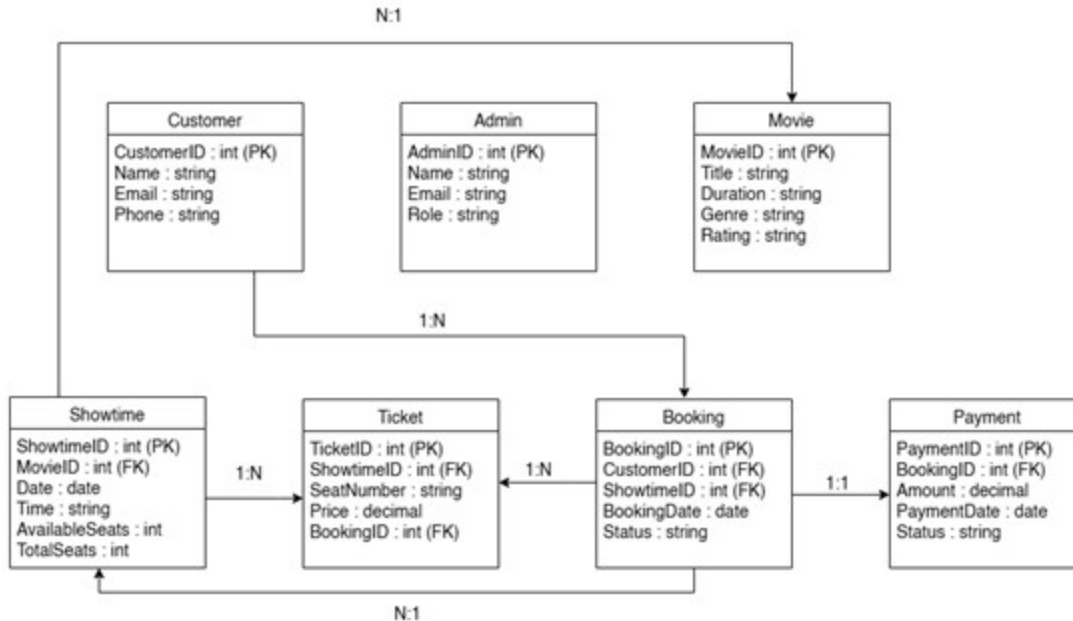
- Identify classes: Customer, Movie, Showtime, Ticket, Booking, Payment, Admin
- Define attributes, including primary keys (PK) and foreign keys (FK)
- Establish relationships and multiplicity between classes
Use these elements in your class diagram:
- **Classes & attributes:**
 - Customer: CustomerID (PK), Name, Email, Phone
 - Movie: MovieID (PK), Title, Duration, Genre, Rating
 - Showtime: ShowtimeID (PK), MovieID (FK), Date, Time, AvailableSeats, TotalSeats
 - Ticket: TicketID (PK), ShowtimeID (FK), SeatNumber, Price
 - Booking: BookingID (PK), CustomerID (FK), ShowtimeID (FK), BookingDate, Status
 - Payment: PaymentID (PK), BookingID (FK), Amount, PaymentDate, Status

- Admin: AdminID (PK), Name, Email, Role

- Relationships with multiplicity:**

- Customer 1:N Booking
- Movie 1:N Showtime
- Showtime 1:N Ticket
- Booking 1:1 Payment

▼ Click here to view the solution:



Part 3: Create a sequence diagram

Why this matters:

A sequence diagram models how objects interact over time to complete a use case. It helps clarify the flow of messages and the order of operations.

Task:

Create a sequence diagram for the "Book Ticket" use case

Instructions:

- Identify objects: Customer, Booking System, Showtime, Payment Gateway, Ticket
- Define the interaction steps from browsing to booking confirmation
- Show synchronous and asynchronous messages
- Include lifelines and activation bars

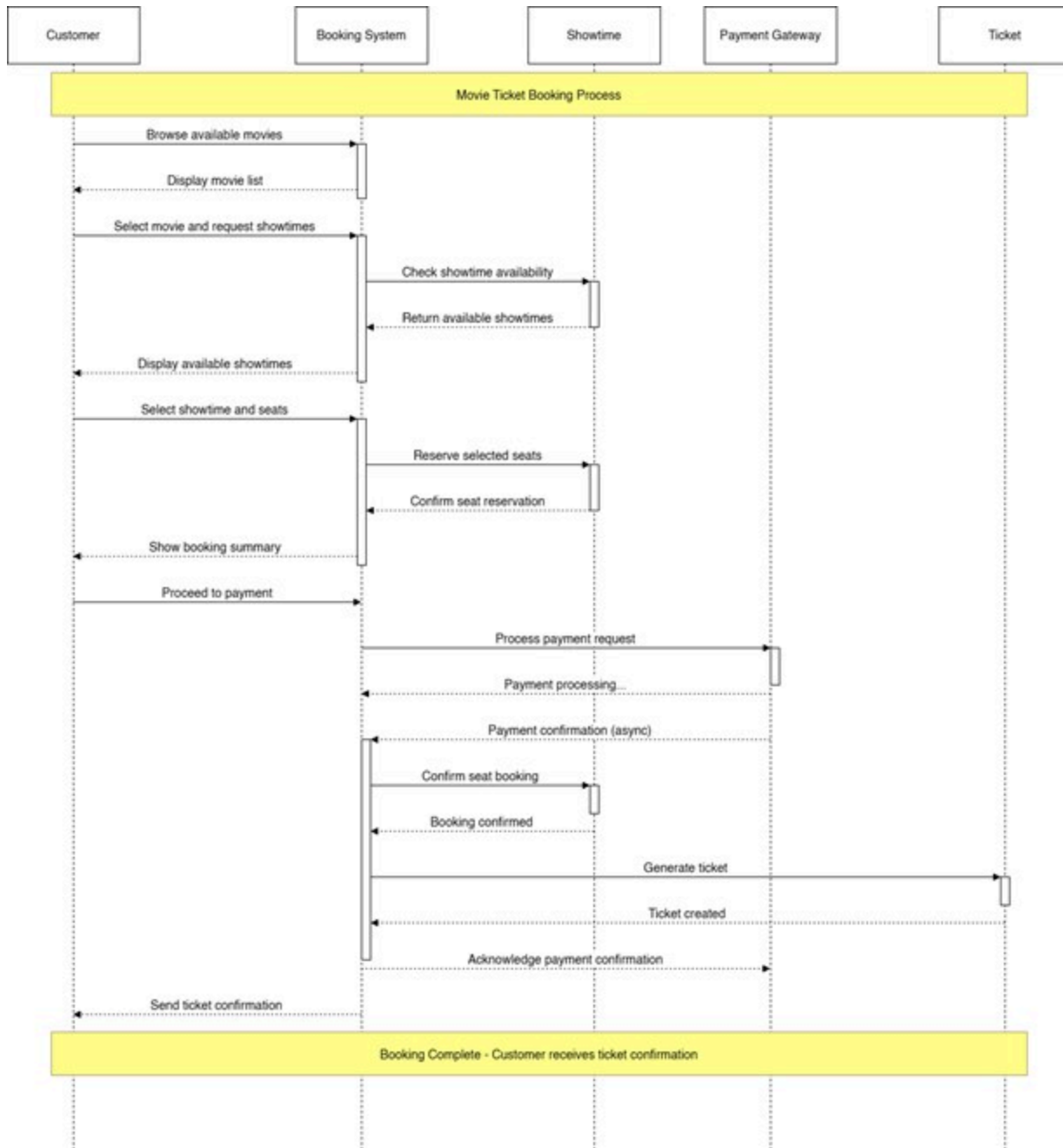
Use these elements in your sequence diagram:

- Objects:** Customer, Booking System, Showtime, Payment Gateway, Ticket
- Flow:**
 - Customer → Booking System: Browse Movies
 - Booking System → Showtime: Check Availability
 - Customer → Booking System: Select Showtime, Book Ticket

4. Booking System → Payment Gateway: Make Payment (async)
5. Payment Gateway → Booking System: Payment Confirmation
6. Booking System → Ticket: Generate Ticket
7. Booking System → Customer: Ticket Confirmation

- **Notation:** Lifelines for each object, activation bars for interaction periods

▼ Click here to view the solution:



Part 4: Create an activity diagram

Why this matters:

An activity diagram visualizes the step-by-step workflow and decision points in a use case. It is useful for identifying logic, branching, and responsibilities.

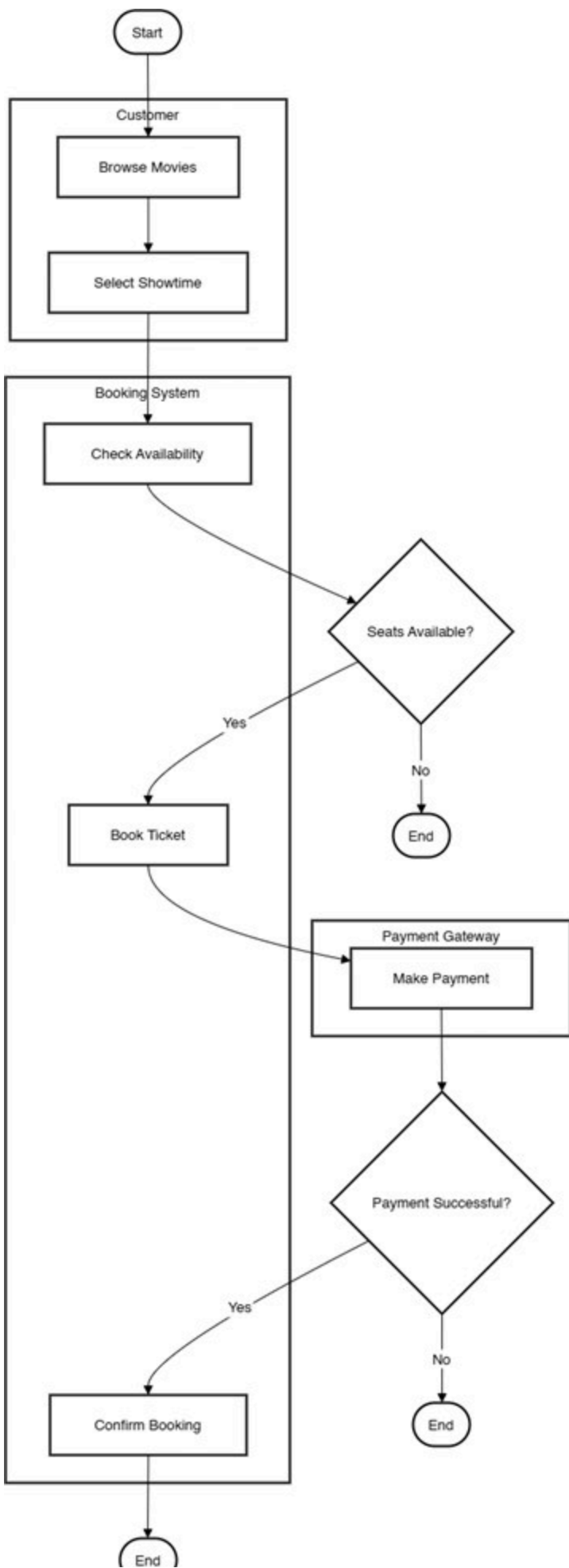
Task:

Create an activity diagram for the "Book Ticket" workflow

Instructions:

- Identify activities: Browse Movies, Select Showtime, Check Availability, Book Ticket, Make Payment, Confirm Booking
- Add decision nodes (e.g., seats available, payment successful)
- Divide responsibilities using swimlanes: Customer, Booking System, Payment Gateway
- Include flow from start to booking confirmation or failure
Use these elements in your activity diagram:
- **Swimlanes:** Customer, Booking System, Payment Gateway
- **Activities:**
 - Customer: Browse Movies, Select Showtime
 - Booking System: Check Availability, Book Ticket, Confirm Booking
 - Payment Gateway: Make Payment
- **Decisions:**
 - Are seats available? → Yes / No → End (if no)
 - Payment successful? → Yes / No → End (if no)
- **Flow:**
 - Start → Browse → Select Showtime → Check Availability → Book Ticket → Make Payment → Confirm Booking → End

▼ [Click here to view the solution:](#)





Summary

In this lab, you designed a UML-based system model for an online movie booking platform. You identified system actors and use cases, defined class structures, mapped dynamic message flows, and charted activity workflows. These diagrams form a complete blueprint of the system, improving clarity for developers, stakeholders, and future maintainers.

Author

Parikshit Jain